

Aula de JavaScript: Arrays e Seus Métodos

1. Métodos de Modificação Básica: push, pop, shift e unshift

push()

Adiciona um ou mais elementos **no final** do array e retorna o novo comprimento.

javascript

```
const frutas = ['maçã', 'banana'];  
frutas.push('laranja');  
console.log(frutas); // ['maçã', 'banana', 'laranja']  
  
frutas.push('uva', 'melancia');  
console.log(frutas); // ['maçã', 'banana', 'laranja', 'uva', 'melancia']
```

pop()

Remove o **último elemento** do array e o retorna.

javascript

```
const numeros = [1, 2, 3, 4];  
const ultimo = numeros.pop();  
console.log(ultimo); // 4  
console.log(numeros); // [1, 2, 3]
```

unshift()

Adiciona um ou mais elementos **no início** do array e retorna o novo comprimento.

javascript

```
const cores = ['verde', 'azul'];  
cores.unshift('vermelho');  
console.log(cores); // ['vermelho', 'verde', 'azul']
```

shift()

Remove o **primeiro elemento** do array e o retorna.

javascript

```
const letras = ['a', 'b', 'c'];  
const primeira = letras.shift();  
console.log(primeira); // 'a'  
console.log(letras); // ['b', 'c']
```



Exercício de Fixação 1

Problema: Crie um array vazio chamado `fila`. Adicione 'João', 'Maria' e 'Pedro' no final da fila. Depois, remova a primeira pessoa da fila e adicione 'Ana' no início. Qual o resultado final?

Resolução:

javascript

```
const fila = [];  
  
// Adiciona pessoas no final  
fila.push('João');  
fila.push('Maria');  
fila.push('Pedro');  
console.log(fila); // ['João', 'Maria', 'Pedro']  
  
// Remove o primeiro da fila  
const removido = fila.shift();  
console.log(removido); // 'João'  
  
// Adiciona Ana no início  
fila.unshift('Ana');  
console.log(fila); // ['Ana', 'Maria', 'Pedro']
```

2. Métodos de Manipulação Avançada: splice, slice, indexOf e includes

splice()

Modifica o array original removendo, substituindo ou adicionando elementos em qualquer posição.

Sintaxe: `array.splice(índiceInicial, quantidadeRemover, elemento1, elemento2, ...)`

javascript

```
const animais = ['gato', 'cachorro', 'pássaro', 'peixe'];
```

```
// Remove 2 elementos a partir do índice 1
```

```
animais.splice(1, 2);
```

```
console.log(animais); // ['gato', 'peixe']
```

```
// Adiciona elementos no índice 1 sem remover nada
```

```
const frutas = ['maçã', 'laranja'];
```

```
frutas.splice(1, 0, 'banana', 'uva');
```

```
console.log(frutas); // ['maçã', 'banana', 'uva', 'laranja']
```

```
// Substitui elementos
```

```
const numeros = [1, 2, 3, 4, 5];
```

```
numeros.splice(2, 2, 10, 20);
```

```
console.log(numeros); // [1, 2, 10, 20, 5]
```

slice()

Retorna uma **cópia superficial** de uma parte do array, sem modificar o original.

Sintaxe: `array.slice(índiceInicial, índiceFinal)`

javascript

```
const cores = ['vermelho', 'verde', 'azul', 'amarelo', 'roxo'];

const algumas = cores.slice(1, 3);
console.log(algumas); // ['verde', 'azul']
console.log(cores); // Array original permanece intacto

const ultimas = cores.slice(-2);
console.log(ultimas); // ['amarelo', 'roxo']
```

indexOf()

Retorna o **primeiro índice** onde o elemento é encontrado, ou -1 se não existir.

```
javascript

const letras = ['a', 'b', 'c', 'b', 'd'];
console.log(letras.indexOf('b')); // 1
console.log(letras.indexOf('z')); // -1

// Pode começar a busca de um índice específico
console.log(letras.indexOf('b', 2)); // 3
```

includes()

Verifica se um elemento existe no array, retornando `true` ou `false`.

```
javascript

const frutas = ['maçã', 'banana', 'laranja'];
console.log(frutas.includes('banana')); // true
console.log(frutas.includes('uva')); // false
```



Exercício de Fixação 2

Problema: Dado o array `['html', 'css', 'javascript', 'python', 'java']`, faça o seguinte:

1. Crie uma cópia com apenas os 3 primeiros elementos

2. Verifique se 'python' está no array original
3. Encontre o índice de 'javascript'
4. Remova 'python' do array original e adicione 'typescript' no seu lugar

Resolução:

javascript

```
const tecnologias = ['html', 'css', 'javascript', 'python', 'java'];

// 1. Cópia dos 3 primeiros
const primeiras = tecnologias.slice(0, 3);
console.log(primeiras); // ['html', 'css', 'javascript']

// 2. Verifica se python existe
const temPython = tecnologias.includes('python');
console.log(temPython); // true

// 3. Índice de javascript
const indiceJS = tecnologias.indexOf('javascript');
console.log(indiceJS); // 2

// 4. Remove python e adiciona typescript
const indicePython = tecnologias.indexOf('python');
tecnologias.splice(indicePython, 1, 'typescript');
console.log(tecnologias); // ['html', 'css', 'javascript', 'typescript', 'java']
```

3. Métodos de Alto Nível: forEach, map, filter e reduce

forEach()

Executa uma função para cada elemento do array. **Não retorna nada** (undefined).

javascript

```
const numeros = [1, 2, 3, 4];

numeros.forEach((numero, indice) => {
  console.log(`Índice ${indice}: ${numero}`);
});
```

```
// Saída:
// Índice 0: 1
// Índice 1: 2
// Índice 2: 3
// Índice 3: 4
```

map()

Cria um **novo array** aplicando uma função a cada elemento do array original.

javascript

```
const numeros = [1, 2, 3, 4];
const dobrados = numeros.map(num => num * 2);
console.log(dobrados); // [2, 4, 6, 8]
console.log(numeros); // [1, 2, 3, 4] - original intacto
```

```
const nomes = ['ana', 'joão', 'maria'];
const maiusculas = nomes.map(nome => nome.toUpperCase());
console.log(maiusculas); // ['ANA', 'JOÃO', 'MARIA']
```

filter()

Cria um **novo array** contendo apenas os elementos que passam em um teste (retornam `true`).

javascript

```
const numeros = [1, 2, 3, 4, 5, 6, 7, 8];
const pares = numeros.filter(num => num % 2 === 0);
console.log(pares); // [2, 4, 6, 8]

const produtos = [
  { nome: 'Notebook', preco: 3000 },
  { nome: 'Mouse', preco: 50 },
  { nome: 'Teclado', preco: 200 }
];

const produtosCaros = produtos.filter(produto => produto.preco > 100);
console.log(produtosCaros);
// [{ nome: 'Notebook', preco: 3000 }, { nome: 'Teclado', preco: 200 }]
```

reduce()

Reduz o array a um **único valor** aplicando uma função acumuladora.

Sintaxe: `array.reduce((acumulador, valorAtual) => { ... }, valorInicial)`

javascript

```
const numeros = [1, 2, 3, 4, 5];
const soma = numeros.reduce((acc, num) => acc + num, 0);
console.log(soma); // 15

// Encontrar o maior número
const maiorNumero = numeros.reduce((maior, num) => {
  return num > maior ? num : maior;
}, numeros[0]);
console.log(maiorNumero); // 5

// Contar ocorrências
const frutas = ['maçã', 'banana', 'maçã', 'laranja', 'banana', 'maçã'];
const contagem = frutas.reduce((acc, fruta) => {
  acc[fruta] = (acc[fruta] || 0) + 1;
  return acc;
}, {});
console.log(contagem); // { maçã: 3, banana: 2, laranja: 1 }
```



Exercício de Fixação 3

Problema: Dado o array de objetos abaixo, realize as seguintes operações:

javascript

```
const alunos = [
  { nome: 'João', nota: 7.5 },
  { nome: 'Maria', nota: 9.0 },
  { nome: 'Pedro', nota: 5.5 },
  { nome: 'Ana', nota: 8.5 }
];
```

1. Use `forEach` para imprimir o nome e nota de cada aluno
2. Use `map` para criar um array apenas com os nomes
3. Use `filter` para obter apenas os alunos aprovados (nota ≥ 7)

4. Use `reduce` para calcular a média das notas

Resolução:

javascript

```
const alunos = [  
  { nome: 'João', nota: 7.5 },  
  { nome: 'Maria', nota: 9.0 },  
  { nome: 'Pedro', nota: 5.5 },  
  { nome: 'Ana', nota: 8.5 }  
];  
  
// 1. forEach - imprime informações  
console.log('=== Lista de Alunos ===');  
alunos.forEach(aluno => {  
  console.log(`${aluno.nome}: ${aluno.nota}`);  
});  
  
// 2. map - array de nomes  
const nomes = alunos.map(aluno => aluno.nome);  
console.log('Nomes:', nomes); // ['João', 'Maria', 'Pedro', 'Ana']  
  
// 3. filter - apenas aprovados  
const aprovados = alunos.filter(aluno => aluno.nota >= 7);  
console.log('Aprovados:', aprovados);  
// [{ nome: 'João', nota: 7.5 }, { nome: 'Maria', nota: 9.0 }, { nome: 'Ana', nota: 8.5 }]  
  
// 4. reduce - média das notas  
const media = alunos.reduce((acc, aluno) => acc + aluno.nota, 0) / alunos.length;  
console.log('Média da turma:', media.toFixed(2)); // 7.62
```

Desafios Práticos

Problema 1: Gerenciador de Carrinho de Compras

Enunciado: Crie um sistema que gerencie um carrinho de compras com as seguintes funcionalidades:

- Adicionar produto ao carrinho
- Calcular o valor total
- Aplicar desconto de 10% para compras acima de R\$ 100

javascript

```
const carrinho = [  
  { produto: 'Camiseta', preco: 49.90, quantidade: 2 },  
  { produto: 'Calça', preco: 89.90, quantidade: 1 },  
  { produto: 'Tênis', preco: 199.90, quantidade: 1 }  
];
```

```
function calcularTotal(itens) {  
  const subtotal = itens.reduce((acc, item) => {  
    return acc + (item.preco * item.quantidade);  
  }, 0);  
  
  const desconto = subtotal > 100 ? subtotal * 0.10 : 0;  
  const total = subtotal - desconto;  
  
  return {  
    subtotal: subtotal.toFixed(2),  
    desconto: desconto.toFixed(2),  
    total: total.toFixed(2)  
  };  
}
```

```
const resultado = calcularTotal(carrinho);  
console.log(`Subtotal: R$ ${resultado.subtotal}`);  
console.log(`Desconto: R$ ${resultado.desconto}`);  
console.log(`Total: R$ ${resultado.total}`);
```

```
// Saída:  
// Subtotal: R$ 389.60  
// Desconto: R$ 38.96  
// Total: R$ 350.64
```

Problema 2: Filtro e Organização de Tarefas

Enunciado: Crie um sistema de gerenciamento de tarefas que filtre e organize por prioridade.

javascript

```
const tarefas = [  
  { id: 1, titulo: 'Estudar JavaScript', concluida: false, prioridade: 'alta' },  
  { id: 2, titulo: 'Fazer exercícios', concluida: true, prioridade: 'média' },  
  { id: 3, titulo: 'Ler documentação', concluida: false, prioridade: 'alta' },  
  { id: 4, titulo: 'Revisar código', concluida: false, prioridade: 'baixa' },  
  { id: 5, titulo: 'Deploy do projeto', concluida: true, prioridade: 'alta' }  
];  
  
function obterTarefasPendentesImportantes(tarefas) {  
  return tarefas  
    .filter(tarefa => !tarefa.concluida && tarefa.prioridade === 'alta')  
    .map(tarefa => tarefa.titulo);  
}  
  
function contarTarefasPorPrioridade(tarefas) {  
  return tarefas.reduce((acc, tarefa) => {  
    const prioridade = tarefa.prioridade;  
    acc[prioridade] = (acc[prioridade] || 0) + 1;  
    return acc;  
  }, {});  
}  
  
const pendentesImportantes = obterTarefasPendentesImportantes(tarefas);  
console.log('Tarefas pendentes de alta prioridade:', pendentesImportantes);  
// ['Estudar JavaScript', 'Ler documentação']  
  
const contagem = contarTarefasPorPrioridade(tarefas);  
console.log('Tarefas por prioridade:', contagem);  
// { alta: 3, média: 1, baixa: 1 }
```

Problema 3: Análise de Vendas Mensais

Enunciado: Analise dados de vendas para encontrar o melhor mês e calcular

estadísticas.

javascript

```
const vendas = [  
  { mes: 'Janeiro', valor: 15000 },  
  { mes: 'Fevereiro', valor: 18000 },  
  { mes: 'Março', valor: 22000 },  
  { mes: 'Abril', valor: 19000 },  
  { mes: 'Maio', valor: 25000 },  
  { mes: 'Junho', valor: 21000 }  
];
```

```
function analisarVendas(vendas) {  
  const totalAnual = vendas.reduce((acc, venda) => acc + venda.valor, 0);  
  
  const mediaMensal = totalAnual / vendas.length;  
  
  const melhorMes = vendas.reduce((melhor, atual) => {  
    return atual.valor > melhor.valor ? atual : melhor;  
  });
```

```
  const mesesAcimaDaMedia = vendas  
    .filter(venda => venda.valor > mediaMensal)  
    .map(venda => venda.mes);
```

```
  return {  
    totalAnual,  
    mediaMensal: mediaMensal.toFixed(2),  
    melhorMes: melhorMes.mes,  
    melhorValor: melhorMes.valor,  
    mesesAcimaDaMedia  
  };  
}
```

```
const analise = analisarVendas(vendas);  
console.log('=== Análise de Vendas ===');  
console.log(`Total Anual: R$ ${analise.totalAnual.toLocaleString('pt-BR')}`);  
console.log(`Média Mensal: R$ ${analise.mediaMensal}`);  
console.log(`Melhor Mês: ${analise.melhorMes} (R$ ${analise.melhorValor.toLocaleString('pt-BR')}`);  
console.log(`Meses acima da média: ${analise.mesesAcimaDaMedia.join(', ')}`);
```

```
// Saída:  
// === Análise de Vendas ===  
// Total Anual: R$ 120.000  
// Média Mensal: R$ 20000.00  
// Melhor Mês: Maio (R$ 25.000)  
// Meses acima da média: Março, Maio, Junho
```

Resumo dos Conceitos

Método	Modifica Original?	Retorno	Uso Principal
push()	✓ Sim	Novo tamanho	Adicionar no final
pop()	✓ Sim	Elemento removido	Remover do final
shift()	✓ Sim	Elemento removido	Remover do início
unshift()	✓ Sim	Novo tamanho	Adicionar no início
splice()	✓ Sim	Elementos removidos	Modificar qualquer posição
slice()	× Não	Novo array	Copiar parte do array
indexOf()	× Não	Índice ou -1	Encontrar posição
includes()	× Não	Boolean	Verificar existência
forEach()	× Não	undefined	Iterar elementos
map()	× Não	Novo array	Transformar elementos
filter()	× Não	Novo array	Filtrar elementos
reduce()	× Não	Valor único	Acumular/agregar

Dicas Finais

1. Use **map** quando precisar transformar todos os elementos
2. Use **filter** quando precisar selecionar alguns elementos
3. Use **reduce** quando precisar combinar elementos em um único valor

4. Use `forEach` quando precisar apenas iterar sem criar novo array
5. **Prefira métodos imutáveis** (que não modificam o original) para código mais previsível
6. **Combine métodos** para operações complexas: `array.filter().map().reduce()`